

724. Find Pivot Index

Easy 🔖 Topics 🏷 Companies 🏢 Hint 📌

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return the **leftmost pivot index**. If no such index exists, return `-1`.

Example 1:

Input: `nums = [1,7,3,6,5,6]`

Output: 3

Explanation:

The pivot index is 3.

Left sum = `nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11`

Right sum = `nums[4] + nums[5] = 5 + 6 = 11`

Example 2:

Input: `nums = [1,2,3]`

Output: -1

Explanation:

There is no index that satisfies the conditions in the problem statement.

Example 3:

Input: `nums = [2,1,-1]`

Output: 0

Explanation:

The pivot index is 0.

Left sum = 0 (no elements to the left of index 0)

Right sum = `nums[1] + nums[2] = 1 + -1 = 0`

```
1 class Solution {
2     public int pivotIndex(int[] nums) {
3         int totalSum = 0;
4         int leftSum = 0;
5
6         // Calculate the total sum of all elements in the array
7         for (int num : nums) {
8             totalSum += num;
9         }
10
11        // Iterate through the array to find the leftmost pivot index
12        for (int i = 0; i < nums.length; i++) {
13            // Right sum is totalSum minus leftSum minus the current element
14            if (leftSum == totalSum - leftSum - nums[i]) {
15                return i;
16            }
17        }
18        // If no pivot index is found, return -1
19        return -1;
20    }
21 }
```

Accepted Runtime: 0 ms

✅ Case 1 ✅ Case 2 ✅ Case 3

Input

`nums =`
`[1,7,3,6,5,6]`

Output

3

Expected

3

Change Theme Language Java 7

```
1 import java.util.*;
2
3 class Student{
4     private int id;
5     private String fname;
6     private double cgpa;
7     public Student(int id, String fname, double cgpa) {
8         super();
9         this.id = id;
10        this.fname = fname;
11        this.cgpa = cgpa;
12    }
13    public int getId() {
14        return id;
15    }
16    public String getFname() {
17        return fname;
18    }
19    public double getCgpa() {
20        return cgpa;
21    }
22 }
23
24 //Complete the code
25 public class Solution
26 {
27     public static void main(String[] args){
28         Scanner in = new Scanner(System.in);
29         int testCases = Integer.parseInt(in.nextLine());
30
31         List<Student> studentList = new ArrayList<Student>();
32         while(testCases>0){
33             int id = in.nextInt();
34             String fname = in.next();
35             double cgpa = in.nextDouble();
36
```

977. Squares of a Sorted Array

Easy Topics Companies

Given an integer array `nums` sorted in **non-decreasing** order, return an array of *the squares of each number* sorted in **non-decreasing** order.

Example 1:

Input: `nums = [-4,-1,0,3,10]`

Output: `[0,1,9,16,100]`

Explanation: After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`

Output: `[4,9,9,49,121]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in **non-decreasing** order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

Java Auto

```
1 class Solution {
2     public int[] sortedSquares(int[] nums) {
3         int n = nums.length;
4         int[] result = new int[n];
5
6         int left = 0;
7         int right = n - 1;
8         int index = n - 1; // Start filling the result array from the back
9
10        while (left <= right) {
11            int leftSquare = nums[left] * nums[left];
12            int rightSquare = nums[right] * nums[right];
13
14            if (leftSquare > rightSquare) {
15                result[index] = leftSquare;
16                left++; // Move the left pointer forward
17            } else {
18                result[index] = rightSquare;
19                right--;
20            }
21            index--;
22        }
23    }
24 }
```

Saved

Ln 27, Col 1

📄 Testcase ➡️ Test Result

Accepted Runtime: 0 ms

✅ Case 1 ✅ Case 2

Input

`nums =`
`[-4,-1,0,3,10]`

Output

`[0,1,9,16,100]`

Expected

`[0,1,9,16,100]`

🤍 Contribute a testcase

👤 119 Online

Seen this question in a real interview before? 1/6

Yes No
👍 10.4K 🗨️ 196 🌟 📄 🕒

```
1  import java.util.*;
2
3  // Write your Checker class here
4
5  class Player{
6      String name;
7      int score;
8
9      Player(String name, int score){
10         this.name = name;
11         this.score = score;
12     }
13 }
14
15 class Solution {
16
17     public static void main(String[] args) {
18         Scanner scan = new Scanner(System.in);
19         int n = scan.nextInt();
20
21         Player[] player = new Player[n];
22         Checker checker = new Checker();
23
24         for(int i = 0; i < n; i++){
25             player[i] = new Player(scan.next(), scan.nextInt());
26         }
27         scan.close();
28
29         Arrays.sort(player, checker);
30         for(int i = 0; i < player.length; i++){
31             System.out.printf("%s %s\n", player[i].name, player[i].score);
32         }
33     }
34 }
```

Line: 2 Col: 1



```
3 class Player {
4     String name;
5     int score;
6
7     Player(String name, int score) {
8         this.name = name;
9         this.score = score;
10    }
11 }
12
13 class Checker implements Comparator<Player> {
14     // complete this method
15     public int compare(Player a, Player b) {
16
17     }
18 }
19
20
21 public class Solution {
22
23     public static void main(String[] args) {
24         Scanner scan = new Scanner(System.in);
25         int n = scan.nextInt();
26
27         Player[] player = new Player[n];
28         Checker checker = new Checker();
29
30         for(int i = 0; i < n; i++){
31             player[i] = new Player(scan.next(), scan.nextInt());
32         }
33         scan.close();
34
35         Arrays.sort(player, checker);
36         for(int i = 0; i < player.length; i++){
37             System.out.printf("%s %s\n", player[i].name, player[i].score);
38         }
39     }
40 }
```

Problem List

1480. Running Sum of 1d Array

Easy Topics Companies Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]..nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`
Output: `[3,4,6,16,17]`

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

Submit

Code

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        for (int i = 1; i < nums.length; i++) {  
            nums[i] += nums[i - 1];  
        }  
        return nums;  
    }  
}
```

Saved

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

`nums =`
`[1,2,3,4]`

Output

`[1,3,6,10]`

Expected

`[1,3,6,10]`

Contribute a testcase

8.9K

222

113 Online

32°C Light rain

Search

Taskbar icons: File Explorer, VS Code, Word, Edge, etc.

ENG IN 11:08 13-08-2026

1672. Richest Customer Wealth

Easy Topics Companies Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the **wealth** that the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: `accounts = [[1,5],[7,3],[3,5]]`
Output: 10
Explanation:
1st customer has wealth = 6
2nd customer has wealth = 10
3rd customer has wealth = 8
The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: `accounts = [[2,8,7],[7,1,3],[1,9,5]]`
Output: 17

Constraints:

$m == accounts.length$

4.9K 110 ☆ 🔍 🔔

Submit

< > Code

```
Java Auto
1 class Solution {
2     public int maximumWealth(int[][] accounts) {
3         int maxWealth = 0;
4
5         for (int i = 0; i < accounts.length; i++) {
6             int currentWealth = 0;
7             for (int j = 0; j < accounts[i].length; j++) {
8                 currentWealth += accounts[i][j];
9             }
10            maxWealth = Math.max(maxWealth, currentWealth);
11        }
12
13        return maxWealth;
14    }
15 }
16
```

Saved

Ln 16, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`accounts =`
`[[1,2,3],[3,2,1]]`

Output

6

Expected

6

Contribute a testcase

36 Online